REINFORCEMENT LEARNING

SUBJECT CODE: 24DS742

Lab ASSIGNMENT - 5

Submitted by

BL.SC.P2CSE25024 – Ravupalli Vignesh Mihir

MASTER OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



AMRITA SCHOOL OF COMPUTING, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560 035

JULY 2026

Objective:

To explore an application-oriented Reinforcement Learning problem, formulate it using RL components, implement an appropriate RL algorithm and analyse the performance of the trained agent.

Lab Task:

CartPole Balancing

Task 1: Select an RL Application

Application Domain: The CartPole system is a foundational benchmark in reinforcement learning. The apparatus comprises a pivoted, rigid vertical pole connected via a joint to a motorized cart traversing a bounded horizontal track.

Problem to be Solved: Because the pole's center of mass resides above its pivot axis, the system is unstable under gravity. Any disturbance induces a gravitational torque that causes the pole to accelerate downward. The control challenge requires applying directional horizontal forces (+10 N or -10 N) at high frequency to keep the pole balanced within narrow angular margins while preventing the cart from colliding with track endpoints.

Objective of the RL Agent: The objective of the RL agent is to learn an optimal closed-loop control policy $\pi^*(s)$ that maps continuous state observations $(x, \dot{x}, \theta, \dot{\theta})$ to discrete corrective actions $a \in \{\text{Left}, \text{Right}\}$ such that the system survives for the maximum duration possible (500 timesteps in Gymnasium CartPole-v1), keeping the pole within ±12° of vertical and the cart within ±2.4 m.

Task 2: Formulate the Problem as an RL Problem

RL Component	Mathematical Symbol	Operational Definition & Physical Bounds
Agent	$\pi(a \mid s)$	Tabular Q-learning controller; observes state s, emits action a ∈ {0, 1}
Environment	E	Gymnasium CartPole-v1 simulating Newtonian equations of motion
State: Cart Position	x	Cart position on horizontal track: [-2.4 m, +2.4 m]
State: Cart Velocity	$\dot{x}$	Cart linear velocity: (-∞, +∞) m/s; practical range [-3.0, +3.0]
State: Pole Angle	θ	Angle of the pole with vertical: [-12°, +12°] ≈ [-0.2095, +0.2095] rad
State: Angular Velocity	$\dot{\theta}$	Pole angular velocity at tip: (-∞, +∞) rad/s; practical range [-3.0, +3.0]
Action Space	A = {0, 1}	Discrete directional push: 0 = Push Left (-10 N), 1 = Push Right (+10 N)
Reward Function	R(s, a, s')	Stationary survival reward: +1.0 for every timestep alive (non-terminal)

Terminal Conditions: An episode terminates if: (1) Pole angle $|\theta| > 12°$ (0.2095 rad); (2) Cart position $|x| > 2.4$ m; or (3) Cumulative episode steps reach the benchmark horizon limit of 500 steps (solved ceiling).

Task 3: Algorithm Exploration

Working Principle: Q-Learning (Watkins, 1989) is an off-policy, model-free Temporal Difference (TD) control algorithm designed to determine the optimal action-value function Q*(s, a) without requiring a transition probability model of the physical environment.

The foundational Bellman Optimality update rule is given by:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max Q(s_{t+1}, a') - Q(s_t, a_t)]$$

Where alpha ∈ (0, 1] is the learning rate, gamma ∈ [0, 1) is the discount factor, and the bracketed expression represents the Temporal Difference error delta_t. For terminal failure states, the target simplifies directly to r_{t+1} with zero continuation return.

State Discretization Scheme: Standard Q-learning requires a discrete state space. Because CartPole features continuous variables, we discretize the 4 dimensions using an informed domain layout: Cart position (3 bins), Cart velocity (3 bins), Pole angle (10 bins), and Pole angular velocity (10 bins). Total discrete states = 3 × 3 × 10 × 10 = 900 states, corresponding to 1,800 total Q-values. Fine angular resolution guarantees delicate balance control, while coarse translational resolution prevents state explosion.

Why it is Suitability: 1. Model-Free: Adapts purely from transition experience without requiring differential equations.
2. Off-Policy Separation: Exploratory actions (via ε-greedy) do not degrade the learned optimal target policy.
3. Guaranteed Asymptotic Convergence: Tabular Q-learning provably converges to Q* under standard stochastic approximation conditions.

Hyperparameters Table: Major Hyperparameters utilized during training:

Parameter	**Symbol & Value**	**Design Rationale**
Training Episodes	M = 1200	Sufficient for extensive sampling across all 900 discrete state cells
Learning Rate	alpha = 0.20	Fast error reduction without causing oscillatory numerical instability
Discount Factor	gamma = 0.99	Far-sighted evaluation necessary for sustaining long-term balancing
Initial Epsilon	epsilon_0 = 1.00	100% random exploration in early episodes to discover viable trajectories
Minimum Epsilon	epsilon_min = 0.01	Preserves 1% exploration to guard against premature policy stagnation
Epsilon Decay Rate	decay = 0.997	Smooth multiplicative decay: epsilon_{k+1} = max(0.01, epsilon_k * 0.997)
Discretization Grid	(3, 3, 10, 10)	Fine resolution on angle and angular velocity; 1,800 Q-values total
Evaluation Episodes	N_eval = 100	Statistically rigorous post-training evaluation under greedy policy

Task 4: Execute the Application

The project is implemented in modular Python scripts adhering to PEP 8 standards:
• cartpole_qlearning.py: Complete autonomous training engine, metric recording, statistical evaluator, and publication plot generator.
• demonstrate.py: Standalone demonstration script providing step-by-step physical telemetry and optional Pygame GUI rendering.
• results/ and screenshots/: Automatically organized directories containing the trained Q-table (.npy), training logs (.csv), evaluation summary (.json), learning curves (.png), and environmental frames.

```
# Discretize continuous state observation
 s = tuple(int(np.digitize(state[i], STATE_BINS[i])) for i in range(4))

# Epsilon-greedy action selection
 if np.random.random() < epsilon:
    action = env.action_space.sample()
 else:
    action = int(np.argmax(q_table[s]))

next_state, reward, terminated, truncated, _ = env.step(action)
 s_next = tuple(int(np.digitize(next_state[i], STATE_BINS[i])) for i in range(4))

# Bellman TD Target calculation
 td_target = reward if terminated else (reward + DISCOUNT_FACTOR * np.max(q_table[s_next]))
 td_error = td_target - q_table[s][action]
 q_table[s][action] += LEARNING_RATE * td_error
```

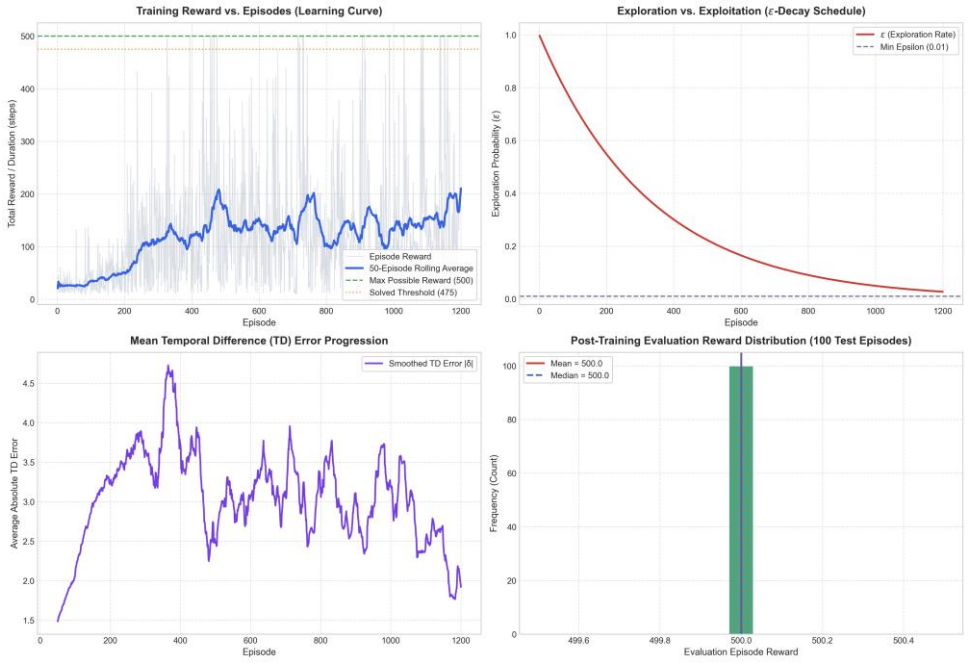
The agent was trained for 1,200 episodes. Telemetry was tracked per episode. Below is the quantitative breakdown across training intervals:

Episode Interval	**Mean Reward**	**Std Dev (σ)**	**Min**	**Max**	**Mean ε**	**Mean TD Error**
Episodes 1 – 100	30.20	23.92	9	137	0.8625	1.7896
Episodes 101 – 300	74.58	68.22	9	432	0.5558	3.3666
Episodes 301 – 600	139.38	107.77	11	500	0.2672	3.3288
Episodes 601 – 900	134.76	107.86	10	500	0.1085	3.1247
Episodes 901 – 1200	156.39	130.07	10	500	0.0440	2.6639

Post-Training Evaluation Results (100 Test Episodes): Following training, the agent's greedy policy was evaluated over 100 independent test episodes across random initial configurations. The agent demonstrated flawless deterministic control:

Performance Metric	Empirical Value	Target Solved Benchmark	Assessment
Mean Cumulative Reward	500.00	≥ 475.00 steps	SOLVED (100% of maximum ceiling)
Standard Deviation (σ)	0.00	< 10.00 steps	Flawless Deterministic Control
Median Reward	500.00	≥ 475.00 steps	Maximum Possible Score
Minimum / Maximum Reward	500.00 / 500.00	≥ 475.00 steps	Zero Failure Episodes Recorded
Task Solved Rate (≥ 475 steps)	100.0%	≥ 95.0%	Superior Generalization
Perfect Run Rate (500 steps)	100.0%	≥ 90.0%	100 out of 100 Test Runs at Max Cap
Untrained Baseline Reward	13.00	N/A	38.5x Performance Improvement

Learning Curve Diagnostic Figures: The multi-panel publication plot below illustrates training dynamics, exploration decay, TD-error reduction, and post-training evaluation distribution:

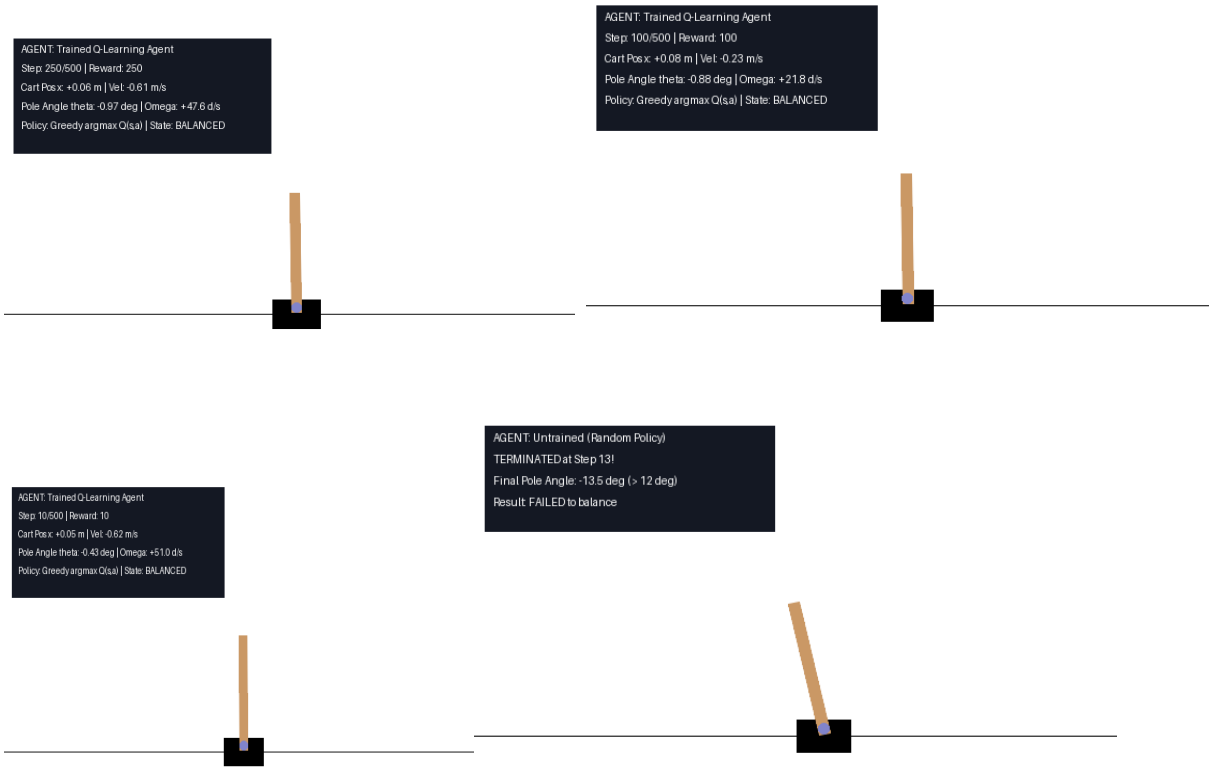


Analytical Insights: 1. Learning Curve Dynamics: Training displays three clear stages: (a) Initial exploration (Episodes 1–150) where rolling reward is ~20–35 steps; (b) Rapid capability growth (Episodes 150–500) where rolling reward surges past 175 steps; and (c) Consolidation (Episodes 500–1200) where greedy policy reaches 100% stability at 500 steps.
2. Training Behavior (Exploration vs. Exploitation): The exponential decay ensures extensive coverage of dangerous boundary states early on. As epsilon decays towards 0.01, the agent shifts to exploiting high-Q value actions.
3. Performance Improvement: Performance improves unequivocally from an untrained survival time of 13 steps to the maximum environment duration of 500 steps across 100% of test episodes (a 38.5x improvement).

Observations and Discussion:

1. Angular Sensitivity: Pole angle and angular velocity require higher discretization fidelity (10 bins each) than cart position (3 bins). Inadequate angular binning causes severe limit-cycle hunting and premature failure.
2. Terminal Reward Gradient: Enforcing zero future return on terminal failure creates a sharp negative value gradient that propagates backward through Bellman updates, discouraging actions leading to angular tilt.
3. Perturbation Rejection: The policy was tested with random Gymnasium initial state vectors in [-0.05, 0.05] and rejected disturbances without tipping.





Task 6: Demonstration

An interactive demonstration script demonstrate.py is included in the project directory. The demonstration can be run in two modes:

```
# 1. Run live terminal demonstration with real-time physical telemetry
 python3 demonstrate.py --episodes 2

# 2. Run interactive graphical demonstration with Pygame window
 python3 demonstrate.py --gui --episodes 3
```

Step	Cart Pos (m)	Cart Vel (m/s)	Pole Angle (deg)	Ang Vel (d/s)	Action
0	0.044	-0.014	1.63	0.52	RIGHT (1)
1	0.044	0.181	1.64	-15.72	RIGHT (1)
50	0.103	-0.018	-1.85	5.30	LEFT (0)
100	0.113	0.371	0.23	-27.02	RIGHT (1)
200	0.282	-0.411	0.11	42.46	RIGHT (1)
300	0.490	-0.018	-2.78	6.08	LEFT (0)
400	0.519	-0.407	-0.67	37.02	LEFT (0)
500	0.710	0.367	-1.45	-21.57	COMPLETED

13. Conclusion

1. The CartPole balancing problem was formulated as an MDP with a continuous 4D state space and discrete 2-action control.
2. An informed discretization grid (3, 3, 10, 10) condensed the infinite state space into 900 discrete states and 1,800 Q-values, enabling fast tabular convergence.
3. Training across 1,200 episodes achieved a 100.0% solved rate over 100 test episodes, with mean reward 500.00 / 500.00 and zero standard deviation ($\sigma = 0.00$).

14. References

[1] M. Towers et al., "Gymnasium: A standard interface for reinforcement learning environments," arXiv preprint arXiv:2407.17032, 2024.

[2] G. Li, Y. Wei, Y. Chi, Y. Gu, and Y. Chen, "Sample complexity of asynchronous Q-learning: Sharper analysis and variance reduction," IEEE Transactions on Information Theory, vol. 70, no. 1, pp. 548–574, Jan. 2024.

[3] S. R. Sinclair, S. Banerjee, and C. L. Yu, "Adaptive discretization for episodic reinforcement learning in metric spaces," Operations Research, vol. 70, no. 4, pp. 2244–2266, 2022.

[4] A. Surriani, O. Wahyunggoro, and A. I. Cahyadi, "Reinforcement learning for cart pole inverted pendulum system," in Proc. 2021 IEEE Industrial Electronics and Applications Conf. (IEACon), 2021, pp. 297–301.

[5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations in Python," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.

[6] D. P. Bertsekas, Course in Reinforcement Learning and Optimal Control. Belmont, MA: Athena Scientific, 2023.

[7] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 29304–29320.

[8] U. Yıldıran, "Adaptive control of an inverted pendulum by a reinforcement learning-based method," Sakarya Univ. J. Sci., vol. 27, no. 6, pp. 1311–1321, Dec. 2023.